

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

APPENDIX G

SELF-RELATIONS | SERVERPOD

 Copy as Markdown

Self-relations

A self-relation (a self-referential relation) occurs when a table has a foreign key that references its own primary key. This creates a relationship between different rows within the same table.

One-to-one

Imagine we have a blog and want to create links between our posts, where you can traverse forward and backward in the post history. Then we can create a self-referencing relation pointing to the next post in the chain.

```
class: Post
table: post
fields:
  content: String
  previous: Post?, relation(name=next_previous_post)
  next: Post?, relation(name=next_previous_post, optional, field=nextId,
onDelete=SetNull)
indexes:
  next_unique_idx:
    fields: nextId
    unique: true
```

In this example, there is a named relation holding the data on both sides of the relation. Serverpod generates the `nextId` field named by `field=`, which stores the id of the next post. The `optional` keyword makes it nullable, because the first post has nothing to point to. The next post represents the object on "this" side while the previous post is the corresponding object on the "other" side. The previous post is the one whose `nextId` points at this post. The `onDelete` behavior is covered in [Referential actions](#).

One-to-many



In a one-to-many self-referenced relation there is one object field connected to a list field. In this example we have modeled the relationship between a cat and her potential kittens. Each cat has at most one mother but can have many kittens. For brevity, we have only modeled the mother.

```
class: Cat
table: cat
fields:
  name: String
  mother: Cat?, relation(name=cat_kittens, optional, onDelete=SetNull)
  kittens: List<Cat>?, relation(name=cat_kittens)
```

The field `motherId: int?` is injected into the Dart class, and it is nullable since we marked the `mother` field as an `optional` relation. A cat's kittens are the rows whose `motherId` equals this cat's `id`. A cat's mother is the row whose `id` equals this cat's `motherId`.

Many-to-many

Imagine a system where members can block other members, and you want to query both who a member is blocking and who is blocking them. This can be achieved by modeling the data as a many-to-many relationship.

Each member has a list of all other members they are blocking and another list of all members that are blocking them. As with any `many-to-many` relation, a junction table holds the connection between the rows.

```
class: Member
table: member
fields:
  name: String
  blocking: List<Blocking>?, relation(name=member_blocked_by_me)
  blockedBy: List<Blocking>?, relation(name=member_blocking_me)
```

```
class: Blocking
table: blocking
fields:
  blocked: Member?, relation(name=member_blocking_me, onDelete=Cascade)
  blockedBy: Member?, relation(name=member_blocked_by_me, onDelete=Cascade)
indexes:
  blocking_blocked_unique_idx:
    fields: blockedId, blockedById
    unique: true
```

The junction table has an entry for who is blocking and another for who is getting blocked. Notice that the `blockedBy` field in the junction table is linked to the `blocking` field in the member table. We have also added a combined unique constraint on `blockedId` and `blockedById`, so a member can block another member only once.

The cascade delete means that if a member is deleted all the blocking entries are also removed for that member.